

Basic Information

Student name: Yaotian Liu

Project title: RTLRepair-Agent: Verification-Guided Agentic RTL Generation

Repository / notebook link: <https://github.com/ytliu74/rtlrepair-agent>

Configuration location: README.md (setup); .env.example (API variables); examples/counter/ (inputs).

Section 1. Problem Definition

The task is to convert a natural-language hardware specification and fixed module interface into synthesizable SystemVerilog for an RTL designer. An externally provided testbench independently evaluates the output. Success requires compilation with exit code zero, successful simulation, and an explicit `TEST_PASS` with no `TEST_FAIL`. Syntax/interface errors, runtime errors, timeouts, mismatches, or a missing pass marker are failures. The research question is whether iterative use of compiler and simulator feedback improves functional pass rate over one-shot LLM generation.

Section 2. Motivation and Project Scope

LLM-generated RTL can contain syntax, interface, timing, and functional errors. Compiler diagnostics and simulation mismatches supply objective evidence for a generate–observe–repair workflow. Open-source Icarus Verilog makes automated evaluation feasible within a semester. The current implementation covers one-shot generation and automatic verification. Semester work will add bounded repair and compare it with this control on additional RTL tasks. Synthesis/PPA feedback is a stretch goal. Physical design, SoC generation, and formal verification are outside the initial scope.

Section 3. Runnable Baseline

Python 3.10+ orchestrates an OpenAI-compatible Chat Completions request, saves the generated module, invokes `iverilog -g2012`, and executes the independent testbench with `vvp`. The prompt includes the specification/interface only; the testbench is withheld. `src/llm_client.py`, `rtl_generator.py`, `simulator.py`, and `baseline.py` implement the pipeline. The command is `./scripts/run_baseline.sh`. One-shot generation is the natural control because it has no opportunity to incorporate verification feedback. **Execution status:** the local EDA fixture is verified; the live LLM run is blocked by missing API key/model configuration. No live model result is claimed. The baseline implements no iterative repair.

Section 4. Test Case and Baseline Output

Input: “Create module `counter` with `clk`, `reset`, `enable`, and 8-bit `count`; on rising edges reset to zero, otherwise increment when enabled, otherwise hold; wrap 255 to 0.” Reset must be synchronous and take priority. The evaluator makes 853 checks, including between-edge stability. Actual hand-written fixture output: `Compilation: PASS; Simulation: PASS; Checks: 853; TEST_PASS`. Seven faulty RTL variants were rejected, supporting the evaluator’s ability to detect these errors. This fixture is not LLM-generated evidence. The actual live attempt reports `Model: (not configured); Compilation: NOT RUN; Simulation: NOT RUN; Functional verification: FAIL` because credentials/model are absent. See `results/baseline_results.json` and `artifacts/baseline_output.txt`. **Required screenshot pending:** `artifacts/baseline_screenshot.png` must show a successful live run before submission. No generated counter is claimed yet.

Section 5. Reproducibility and Run Instructions

Clone the public repository and enter `rtlrepair-agent`. Install Python 3.10+ and Icarus Verilog (`brew install icarus-verilog` on macOS; `sudo apt-get install iverilog python3-venv` on Ubuntu). Run `python3 -m venv .venv`, `source .venv/bin/activate`, and `python -m pip install -r requirements.txt` (no third-party Python packages). Copy `.env.example` to `.env`; set `OPENAI_API_KEY` and `OPENAI_MODEL` to an accessible model. `OPENAI_BASE_URL` is optional and defaults to OpenAI's /v1 API root. Run `./scripts/run_baseline.sh`. Inputs are `examples/counter/spec.txt` and `tb.sv`. Outputs are `generated/counter.sv`, `results/baseline_results.json`, and `artifacts/baseline_output.txt`. Configuration is loaded automatically; no secrets are committed. API access is required; no success is guaranteed. Offline tests: `python -m unittest discover -s tests -v`.

Section 6. Initial Evaluation Plan

Compare one-shot generation with verification-guided repair using the same model, initial prompt, tasks, and independent testbenches. Set a repair-call limit and report both single-call performance and improvement per additional call; include budget-matched independent resampling to separate feedback benefit from extra compute. Primary metric: functional pass rate across tasks and repeated trials. Secondary metrics: compilation rate, iterations, LLM calls, token usage, latency, and measurable API cost. Begin with curated small modules; consider VerilogEval and RTLLM later. Keep failed runs and report all trials.

Section 7. Limitations and Next Steps

One example cannot establish generality. Correctness depends on testbench coverage; simulation cannot guarantee untested behavior or synthesizability. Model nondeterminism and API dependency affect repeatability. The current system cannot repair errors, and live generation remains unverified. Immediate steps are to configure the API, run and preserve the live result, capture its screenshot, and update this proposal with observed model/results. Then implement failure diagnosis, bounded repair, broader evaluation, and optional synthesis/PPA feedback.